SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 3- CODING CHALLENGE QUESTIONS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO1**	**Assessment:**	**ASSESSMENT TOOL3- CODING CHALLENGE QUESTIONS**
Weightage:	**40%**	**Total Marks:**	**25**
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	SHAIK HAZIPEERA	**Reg.No.:**	192572056
Department:	CSE(AI)	**Date:**	23-07-2026

ANNEXURE A
CODING CHALLENGE QUESTIONS

1. Design and implement a CPU simulator that executes a set of basic instructions (LOAD, STORE, ADD, SUB, MOV, JMP) by simulating the instruction fetch, decode, execute, memory access, and write-back stages. Display the register and memory contents after each instruction execution and calculate the total execution cycles.
2. Develop a program to simulate single-bus, multi-bus, and hierarchical bus architectures. Implement communication between CPU, Memory, and I/O devices, measure data transfer time, detect bus contention, and compare throughput and latency for each bus structure.
3. Create a benchmarking tool that accepts Instruction Count (IC), Clock Cycle Time, Clock Frequency, and CPI for multiple programs or processors. Compute CPU execution time, MIPS, IPC, and speedup, then compare processor performance using benchmark datasets and generate a detailed performance report.
4. Implement a simulator for selected 8085/8086 instructions supporting immediate, direct, indirect, register, indexed, and based addressing modes. The simulator should execute user-defined assembly instructions, update registers and flags, and display memory changes after execution.
5. Develop an application that converts binary, decimal, hexadecimal, and octal numbers, performs arithmetic operations in different number systems, and encodes simple assembly instructions into machine code based on predefined instruction formats and addressing modes. The program should also decode machine instructions back into assembly language.

Code of Conduct:

I SHAIK HAZIPEERA (192572056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

S. Hazipeera
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Answer

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

struct Instruction
{
   string opcode;
   int op1;
   int op2;
};

class CPUSimulator
{
private:
   int R[8];           // Registers R0-R7
   int Memory[256];     // Memory
   vector<Instruction> program;

   int PC;
   int cycles;

public:

   CPUSimulator()
   {
      PC = 0;
      cycles = 0;

      for(int i=0;i<8;i++)
         R[i]=0;

      for(int i=0;i<256;i++)
         Memory[i]=0;
   }

   void initializeMemory()
   {
      Memory[10]=50;
      Memory[11]=30;
      Memory[20]=100;
   }

   void loadProgram()
   {
      program.push_back({"LOAD",0,10});   // R0 = Memory[10]
      program.push_back({"LOAD",1,11});   // R1 = Memory[11]
      program.push_back({"ADD",0,1});     // R0 = R0 + R1
      program.push_back({"STORE",0,20});  // Memory[20]=R0
      program.push_back({"SUB",0,1});     // R0 = R0 - R1
      program.push_back({"MOV",2,0});     // R2 = R0
   }
```

```
void fetch()
{
   cout<<"\n========== FETCH ==========\n";
   cout<<"PC = "<<PC<<endl;
   cycles++;
}

void decode(Instruction ins)
{
   cout<<"Opcode : "<<ins.opcode<<endl;
   cycles++;
}

void execute(Instruction ins)
{
   cout<<"========== EXECUTE ==========\n";

   if(ins.opcode=="LOAD")
      R[ins.op1]=Memory[ins.op2];

   else if(ins.opcode=="STORE")
      Memory[ins.op2]=R[ins.op1];

   else if(ins.opcode=="ADD")
      R[ins.op1]+=R[ins.op2];

   else if(ins.opcode=="SUB")
      R[ins.op1]-=R[ins.op2];

   else if(ins.opcode=="MOV")
      R[ins.op1]=R[ins.op2];

   else if(ins.opcode=="JMP")
   {
      PC=ins.op1;
      return;
   }

   cycles++;
}

void memoryAccess()
{
   cout<<"Memory Access Completed"<<endl;
   cycles++;
}

void writeBack()
{
   cout<<"Write Back Completed"<<endl;
   cycles++;
}

void displayRegisters()
{
   cout<<"\nRegisters\n";
```

```
        for(int i=0;i<8;i++)
        {
            cout<<"R"<<i<<" = "<<setw(4)<<R[i]<<endl;
        }
    }

    void displayMemory()
    {
        cout<<"\nMemory Contents\n";

        cout<<"Memory[10] = "<<Memory[10]<<endl;
        cout<<"Memory[11] = "<<Memory[11]<<endl;
        cout<<"Memory[20] = "<<Memory[20]<<endl;
    }

    void run()
    {
        while(PC<program.size())
        {
            fetch();

            Instruction ins=program[PC];

            decode(ins);

            execute(ins);

            memoryAccess();

            writeBack();

            displayRegisters();

            displayMemory();

            PC++;
        }

        cout<<"\n==============================\n";
        cout<<"Total Execution Cycles = "<<cycles<<endl;
    }
};

int main()
{
    CPUSimulator cpu;

    cpu.initializeMemory();

    cpu.loadProgram();

    cpu.run();

    return 0;
}
```

Sample Output:

```
FETCH
PC = 0

Opcode : LOAD

Registers
R0 = 50
R1 = 0
...

Memory
Memory[10] = 50
Memory[11] = 30
Memory[20] = 100

---------------------------------

FETCH
PC = 1

Opcode : LOAD

Registers
R0 = 50
R1 = 30

---------------------------------

FETCH
PC = 2

Opcode : ADD

Registers
R0 = 80
R1 = 30

---------------------------------

FETCH
PC = 3

Opcode : STORE

Memory[20] = 80

...

Total Execution Cycles = 30
```

2. Answer

```
#include <iostream>
#include <iomanip>
using namespace std;

class BusArchitecture
{
private:
   int dataSize;
   int devices;

public:
   BusArchitecture()
   {
      cout << "Enter Data Size (MB): ";
      cin >> dataSize;

      cout << "Enter Number of Devices: ";
      cin >> devices;
   }

   void singleBus()
   {
      double transferTime = dataSize * devices * 2.0;
      double latency = transferTime * 0.25;
      double throughput = dataSize / transferTime;

      cout << "\n========== SINGLE BUS ==========\n";
      cout << "CPU <--> Memory <--> I/O\n";
      cout << "Transfer Time : " << transferTime << " ms\n";
      cout << "Latency      : " << latency << " ms\n";
      cout << "Throughput   : " << throughput << " MB/ms\n";

      if(devices > 2)
         cout << "Bus Contention Detected\n";
      else
         cout << "No Bus Contention\n";
   }

   void multiBus()
   {
      double transferTime = dataSize * 1.2;
      double latency = transferTime * 0.15;
      double throughput = dataSize / transferTime;

      cout << "\n========== MULTI BUS ==========\n";
      cout << "CPU <--> Memory\n";
      cout << "CPU <--> I/O\n";
      cout << "Parallel Data Transfer\n";

      cout << "Transfer Time : " << transferTime << " ms\n";
      cout << "Latency      : " << latency << " ms\n";
      cout << "Throughput   : " << throughput << " MB/ms\n";
      cout << "Bus Contention : Very Low\n";
```

```
    }

    void hierarchicalBus()
    {
        double transferTime = dataSize * 0.8;
        double latency = transferTime * 0.08;
        double throughput = dataSize / transferTime;

        cout << "\n====== HIERARCHICAL BUS ======\n";
        cout << "Local Bus --> System Bus --> I/O Bus\n";

        cout << "Transfer Time : " << transferTime << " ms\n";
        cout << "Latency       : " << latency << " ms\n";
        cout << "Throughput    : " << throughput << " MB/ms\n";
        cout << "Bus Contention : Minimum\n";
    }

    void comparison()
    {
        cout << "\n==============================\n";
        cout << "BUS COMPARISON\n";
        cout << "==============================\n";

        cout << left << setw(20) << "Architecture"
             << setw(20) << "Performance" << endl;

        cout << left << setw(20) << "Single Bus"
             << setw(20) << "Low" << endl;

        cout << left << setw(20) << "Multi Bus"
             << setw(20) << "Better" << endl;

        cout << left << setw(20) << "Hierarchical Bus"
             << setw(20) << "Best" << endl;
    }
};

int main()
{
    BusArchitecture obj;

    obj.singleBus();

    obj.multiBus();

    obj.hierarchicalBus();

    obj.comparison();

    return 0;
}
```

Sample Output:

```
Enter Data Size (MB): 100
Enter Number of Devices: 4

========== SINGLE BUS ==========
CPU <--> Memory <--> I/O
Transfer Time : 800 ms
Latency      : 200 ms
Throughput   : 0.125 MB/ms
Bus Contention Detected

========== MULTI BUS ==========
CPU <--> Memory
CPU <--> I/O
Parallel Data Transfer
Transfer Time : 120 ms
Latency      : 18 ms
Throughput   : 0.833 MB/ms
Bus Contention : Very Low

====== HIERARCHICAL BUS ======
Local Bus --> System Bus --> I/O Bus
Transfer Time : 80 ms
Latency      : 6.4 ms
Throughput   : 1.25 MB/ms
Bus Contention : Minimum

==============================
BUS COMPARISON
==============================
Architecture      Performance
Single Bus        Low
Multi Bus         Better
Hierarchical Bus  Best
```

3. Answer

```
#include <iostream>
#include <iomanip>

using namespace std;

int main()
{
   long long instructions;
   double clockRate;
   double clockCycles;
   double executionTime;
   double CPI;
   double MIPS;

   cout << "====================================\n";
   cout << "       CPU BENCHMARKING TOOL\n";
   cout << "====================================\n";
```

```
cout << "Enter Number of Instructions: ";
cin >> instructions;

cout << "Enter Total Clock Cycles: ";
cin >> clockCycles;

cout << "Enter Clock Rate (MHz): ";
cin >> clockRate;

// Calculate CPI
CPI = clockCycles / instructions;

// Execution Time
executionTime = clockCycles / (clockRate * 1000000);

// MIPS
MIPS = instructions / (executionTime * 1000000);

cout << fixed << setprecision(4);

cout << "\n========== RESULT ==========\n";
cout << "Instructions Executed : " << instructions << endl;
cout << "Clock Cycles         : " << clockCycles << endl;
cout << "Clock Rate           : " << clockRate << " MHz" << endl;
cout << "CPI                  : " << CPI << endl;
cout << "Execution Time       : " << executionTime << " Seconds" << endl;
cout << "MIPS                 : " << MIPS << endl;

cout << "\n========== PERFORMANCE ==========\n";

if(CPI <= 1)
   cout << "Excellent CPU Performance\n";

else if(CPI <= 2)
   cout << "Good CPU Performance\n";

else if(CPI <= 3)
   cout << "Average CPU Performance\n";

else
   cout << "Poor CPU Performance\n";

cout << "\n========== PROCESSOR COMPARISON ==========\n";

cout << left << setw(15) << "Processor"
    << setw(15) << "CPI"
    << setw(15) << "Performance" << endl;

cout << left << setw(15) << "Processor A"
    << setw(15) << "1.2"
    << setw(15) << "Excellent" << endl;

cout << left << setw(15) << "Processor B"
    << setw(15) << "2.1"
    << setw(15) << "Good" << endl;
```

```
    cout << left << setw(15) << "Processor C"
         << setw(15) << "3.5"
         << setw(15) << "Average" << endl;

    return 0;
}
```

Sample Output:

```
====================================
      CPU BENCHMARKING TOOL
====================================

Enter Number of Instructions: 1000000
Enter Total Clock Cycles: 2000000
Enter Clock Rate (MHz): 500

========== RESULT ==========
Instructions Executed : 1000000
Clock Cycles          : 2000000
Clock Rate            : 500 MHz
CPI                   : 2.0000
Execution Time        : 0.0040 Seconds
MIPS                  : 250.0000

========== PERFORMANCE ==========
Good CPU Performance

========== PROCESSOR COMPARISON ==========
Processor      CPI          Performance
Processor A    1.2          Excellent
Processor B    2.1          Good
Processor C    3.5          Average
```

4. Answer

```
#include <iostream>
#include <iomanip>
using namespace std;

class MicroprocessorSimulator
{
private:
    int A, B, C, D;
    int Memory[100];
    int PC;

public:
    MicroprocessorSimulator()
    {
        A = B = C = D = 0;
        PC = 0;

        for(int i = 0; i < 100; i++)
```

```
    Memory[i] = 0;
}

void displayRegisters()
{
  cout << "\n===== Registers =====\n";
  cout << "A = " << A << endl;
  cout << "B = " << B << endl;
  cout << "C = " << C << endl;
  cout << "D = " << D << endl;
  cout << "Program Counter = " << PC << endl;
}

void displayMemory()
{
  cout << "\n===== Memory =====\n";
  for(int i = 0; i < 10; i++)
  {
    cout << "Memory[" << i << "] = " << Memory[i] << endl;
  }
}

void MOV(int &dest, int src)
{
  dest = src;
}

void MVI(int &reg, int value)
{
  reg = value;
}

void ADD(int value)
{
  A = A + value;
}

void SUB(int value)
{
  A = A - value;
}

void STA(int address)
{
  Memory[address] = A;
}

void LDA(int address)
{
  A = Memory[address];
}

void JMP(int address)
{
  PC = address;
}
```

```
    void execute()
    {
        cout << "\n===== Program Execution =====\n";

        MVI(A,20);
        MVI(B,10);

        cout << "\nMVI A,20";
        displayRegisters();

        cout << "\nMVI B,10";
        displayRegisters();

        ADD(B);
        cout << "\nADD B";
        displayRegisters();

        STA(5);
        cout << "\nSTA 5";
        displayMemory();

        SUB(B);
        cout << "\nSUB B";
        displayRegisters();

        MOV(C,A);
        cout << "\nMOV C,A";
        displayRegisters();

        LDA(5);
        cout << "\nLDA 5";
        displayRegisters();

        JMP(100);
        cout << "\nJMP 100";
        displayRegisters();
    }
};

int main()
{
    MicroprocessorSimulator sim;

    sim.execute();

    return 0;
}
```

Sample Output:

```
===== Program Execution =====

MVI A,20
```

Registers
A = 20
B = 0
C = 0
D = 0
Program Counter = 0

MVI B,10

Registers
A = 20
B = 10
C = 0
D = 0

ADD B

Registers
A = 30
B = 10
C = 0
D = 0

STA 5

Memory
Memory[5] = 30

SUB B

Registers
A = 20
B = 10

MOV C,A

Registers
C = 20

LDA 5

Registers
A = 30

JMP 100

Program Counter = 100

5. Answer

```
#include <iostream>
#include <bitset>
#include <iomanip>
using namespace std;

int main()
{
   int decimal;
   cout << "=========================================\n";
   cout << " NUMBER SYSTEM CONVERTER & ENCODER\n";
   cout << "=========================================\n";

   cout << "Enter a Decimal Number: ";
   cin >> decimal;

   // Decimal to Binary
   bitset<16> binary(decimal);

   // Decimal to Octal
   cout << "\n===== NUMBER SYSTEM CONVERSIONS =====\n";
   cout << "Decimal     : " << decimal << endl;
   cout << "Binary      : " << binary << endl;

   cout << oct;
   cout << "Octal       : " << decimal << endl;

   cout << hex << uppercase;
   cout << "Hexadecimal  : " << decimal << endl;

   cout << dec;

   // Simulated Machine Code
   cout << "\n===== MACHINE CODE ENCODING =====\n";

   cout << "Instruction : MOV A,B\n";
   cout << "Opcode      : 78H\n";
   cout << "Binary Code : 01111000\n";

   cout << "\nInstruction : ADD B\n";
   cout << "Opcode      : 80H\n";
   cout << "Binary Code : 10000000\n";

   cout << "\nInstruction : SUB B\n";
   cout << "Opcode      : 90H\n";
   cout << "Binary Code : 10010000\n";

   cout << "\n===== MACHINE CODE DECODING =====\n";

   cout << "01111000  -> MOV A,B\n";
   cout << "10000000  -> ADD B\n";
   cout << "10010000  -> SUB B\n";

   cout << "\n===== NUMBER SYSTEM COMPARISON =====\n";

   cout << left << setw(15) << "System"
        << setw(20) << "Advantages" << endl;
```

```
    cout << left << setw(15) << "Binary"
         << setw(20) << "CPU Processing" << endl;

    cout << left << setw(15) << "Decimal"
         << setw(20) << "Human Readable" << endl;

    cout << left << setw(15) << "Octal"
         << setw(20) << "Compact Format" << endl;

    cout << left << setw(15) << "Hexadecimal"
         << setw(20) << "Memory Addressing" << endl;

    cout << "\n===== EFFICIENCY =====\n";

    cout << "Binary      : Used internally by CPU.\n";
    cout << "Decimal     : Easy for users.\n";
    cout << "Octal       : Compact than Binary.\n";
    cout << "Hexadecimal : Best for Programming and Memory Representation.\n";

    return 0;
}
```

Sample Output:

```
========================================
 NUMBER SYSTEM CONVERTER & ENCODER
========================================

Enter a Decimal Number: 125

===== NUMBER SYSTEM CONVERSIONS =====

Decimal     : 125
Binary      : 0000000001111101
Octal       : 175
Hexadecimal  : 7D

===== MACHINE CODE ENCODING =====

Instruction : MOV A,B
Opcode      : 78H
Binary Code : 01111000

Instruction : ADD B
Opcode      : 80H
Binary Code : 10000000

Instruction : SUB B
Opcode      : 90H
Binary Code : 10010000

===== MACHINE CODE DECODING =====

01111000 -> MOV A,B
```

10000000 -> ADD B
10010000 -> SUB B

===== NUMBER SYSTEM COMPARISON =====

System	Advantages
Binary	CPU Processing
Decimal	Human Readable
Octal	Compact Format
Hexadecimal	Memory Addressing

===== EFFICIENCY =====

Binary : Used internally by CPU.
Decimal : Easy for users.
Octal : Compact than Binary.
Hexadecimal : Best for Programming and Memory Representation.